PIRTM–ACE×PETC: Prime-Indexed Recursive Tensor Models with Certified Contraction and Lawful Prime Weighting

IFMD Framework: ACE (Tyler Van Osdol) & PETC (Ryan Van Gelder)

November 13, 2025

Abstract

We present a complete, engineering-ready formulation of the Prime-Indexed Recursive Tensor Model (PIRTM) integrated with the Arithmetic Control Engine (ACE) for safety certificates and the Prime-Encoded Tensor Calculus (PETC) for lawful prime-weighted structure. The core dynamics are affine-linear with a prime-decomposed gain operator. We prove necessary and sufficient convergence conditions (spectral radius < 1), provide sufficient ACE–PETC budget tests, give robustness and time-varying extensions, and supply drop-in Python implementations including exact weighted-ℓ_1 projection, monitoring utilities, fixed-point estimators with certified tails, and adaptive margin control. Throughout we adhere to the notational convention that the gain acts by operator application KT (*not* tensor product), reserving $\otimes$ for true space tensor products.

Contents

1 Introduction **2**

2 Notation and Setting **2**

3 Core Recurrence: Scalar and Operator Forms **3**
3.1 Scalar/Mode-wise Form . . . 3
3.2 Operator Form . . . 3

4 Convergence Theory **3**
4.1 ACE–PETC Sufficient Test . . . 3
4.2 Infinite-prime extension . . . 4

5 Robustness and Time-Varying Systems **4**

6 ACE×PETC Integration **4**

7 Algorithms and Reference Implementations **4**
7.1 Exact weighted-ℓ_1 projection (ACE) . . . 4
7.2 Unified ACE–PETC update (scalar and operator cases) . . . 5
7.3 Monitoring and safety status . . . 6
7.4 Fixed-point estimator with certified tail . . . 7
7.5 Adaptive budget to maintain a target gap . . . 7
7.6 Infinite-prime convergence check (corrected) . . . 7
7.7 Usage example (scalar case) . . . 8
7.8 Design Guidelines and Checklists . . . 8
7.9 Conclusions . . . 8

8 Lightweight Scalar Friendly **9**
8.1 Notation and Setup . . . 9
8.2 Convergence and Safety . . . 9
8.3 PETC Typing: Prime Structure as Metadata . . . 10
8.4 Algorithm (ACE×PETC-compliant) . . . 10
8.5 Practical Guidance . . . 10
8.6 Experimental Protocol (Preregistered) . . . 10
8.7 Related Work . . . 10
8.8 Limitations . . . 11
8.9 Future Work . . . 11

9 The Prime-Recursive Foundations of Mathematical Existence **11**
9.1 The Meta-Recursive Function (ACE×PETC Form) . . . 11
9.2 PETC: Prime-Signature Ledger for Unambiguous Retrospection . . . 12
9.3 Interpretation as a Universal Constructor . . . 12
9.4 Examples of Emergent Structures . . . 12
9.5 Refinements: Compression, Hierarchies, Causality, and Performance . . . 12
9.6 ACE: Exact Weighted-ℓ_1 Projection . . . 13
9.7 Kernel-Level Multiplicity Scheduler . . . 13
9.8 Algorithms and Code (Runnable Sketches) . . . 13
9.9 Operational Checklist . . . 15

A Handy Design Bound for $|k|$ **15**

B Prime-Recursive Mechanical Stabilization: PETC × ACE Formalization **16**
B.1 Prime-Signature Ledger (PETC) for Unambiguous Retrospection . . . 16
B.2 Prime-Indexed Affine Recursion (ACE-Certified) . . . 16
B.3 Algorithms and Code Snippets . . . 17
B.4 Design Pattern and Examples . . . 19
B.5 Takeaways (Operational Rules) . . . 19

1 Introduction

PIRTM models the evolution of a tensor-valued state T_t under a prime-weighted linear operator with an affine forcing term. To guarantee safe and predictable behavior, we combine:

- **PETC**: a structural layer that encodes prime indices, exponents and operator bounds b_p, producing a lawful ledger for $\{(p, \alpha, b_p)\}$ and raw weights;
- **ACE**: a certification layer that enforces a contraction budget via exact weighted-ℓ_1 projection and yields online certificates (contraction gap, rate upper bounds).

This report consolidates the mathematical foundation (convergence, robustness) and the production-grade implementation (projection, monitoring and solvers).

2 Notation and Setting

Let $(H, \|\cdot\|)$ be a Banach space suitable for the tensor $T_t \in H$. For a list of primes $P_N = \{p_1, \ldots, p_N\}$ and bounded linear maps $B_p : H \to H$ with operator bounds $\|B_p\| \le b_p$, define prime weights

$$w_p := \Lambda_p \, p^{\alpha}, \qquad \alpha \in \mathbb{R}, \ \Lambda_p \in \mathbb{R}.$$

The gain operator is the (finite) sum

$$K := \sum_{p \in P_N} w_p \, B_p, \qquad \text{acting by application } (K\,X) = \sum_p w_p \, B_p(X).$$

We reserve $\otimes$ for explicit tensor products of spaces; we *do not* write $K \otimes T$ for the state update.

3 Core Recurrence: Scalar and Operator Forms

3.1 Scalar/Mode-wise Form

For each mode (m, n), the state obeys

$$T_{t+1}(m,n) = k_m \, T_t(m,n) + F(m,n), \qquad k_m := \sum_{p \in P_N} \Lambda_m \, p^\alpha. \tag{1}$$

This is a specialization of the operator model when each B_p acts as the identity on the (m, n)-mode.

3.2 Operator Form

For the full tensor state $T_t \in H$ we have

$$T_{t+1} = F + K\,T_t, \qquad K = \sum_{p \in P_N} w_p \, B_p, \quad w_p = \Lambda_p \, p^\alpha. \tag{2}$$

4 Convergence Theory

Theorem 1 (Fixed point and convergence)**.** *Consider* (2)*. The following are equivalent:*

(i) The spectral radius satisfies $\rho(K) < 1$.

(ii) The operator $(I - K)$ is invertible with Neumann series $(I - K)^{-1} = \sum_{j \geq 0} K^j$ converging in operator norm.

(iii) For every $T_0 \in H$, the sequence $T_{t+1} = F + K\,T_t$ converges to the unique fixed point $T_\infty = (I - K)^{-1} F$ with linear rate bounded by $\|K\|^t$.

Proof. $(i) \Rightarrow (ii)$: If $\rho(K) < 1$, then there exists a submultiplicative operator norm with $\|K\| < 1$, which yields the Neumann series. $(ii) \Rightarrow (iii)$: Summation of $K^j F$ and the Banach fixed-point theorem give uniqueness and linear convergence. $(iii) \Rightarrow (i)$: If iterates converge for all T_0 then 1 is not in the spectrum of K and $\rho(K) < 1$. □

Corollary 2 (Scalar specialization)**.** *If each $B_p = \mathrm{Id}$, then $K = k\,\mathrm{Id}$ with $k = \sum_p w_p$. Convergence holds iff $|k| < 1$, and the fixed point and rate are*

$$T_\infty = \frac{F}{1-k}, \qquad |T_t - T_\infty| \leq |k|^t \, |T_0 - T_\infty|.$$

4.1 ACE–PETC Sufficient Test

A practical sufficient condition enforced by ACE with PETC bounds is

$$\sum_{p \in P_N} b_p \, |w_p| \;<\; 1 - \varepsilon, \qquad (\varepsilon > 0), \tag{3}$$

which implies $\|K\| \leq \sum_p b_p |w_p| < 1$ and hence $\rho(K) < 1$.

4.2 Infinite-prime extension

If $P_N \uparrow P$ and $\sum_{p\in P} b_p\,|\Lambda_p|\,p^\alpha < 1$ (e.g., $\alpha < -1$ with bounded Λ_p and b_p), then the series for K converges absolutely in operator norm and all results above hold.

5 Robustness and Time-Varying Systems

Proposition 3 (Sensitivity of the fixed point)**.** *Let $\|K\| \le 1-\delta$ with $\delta \in (0,1)$. For perturbations ΔK and ΔF with $\|\Delta K\| < \delta$, the fixed point shifts by*

$$\Delta T_\infty = (I-K)^{-1}\,\Delta F + (I-K)^{-1}\,\Delta K\,(I-K)^{-1}F + \mathcal{O}(\|\Delta K\|^2),$$

so that $\|\Delta T_\infty\| \le \delta^{-1}\,\|\Delta F\| + \delta^{-2}\,\|\Delta K\|\;\|F\| + \dots$.

Theorem 4 (Uniform contraction for time-varying gains)**.** *Suppose $T_{t+1} = F_t + K_t T_t$ with $\sup_t \|K_t\| \le 1-\delta$ and bounded F_t. Then*

$$\|T_t\| \le (1-\delta)^t\;\|T_0\| + \frac{1}{\delta}\sup_{j\le t}\|F_j\|\,,$$

so the system is BIBO-stable and tracks slowly varying fixed points with error $\mathcal{O}(\delta^{-1})$.

6 ACE×PETC Integration

PETC (lawfulness). PETC supplies the prime-signature ledger $\{(p,\alpha,b_p)\}$ and raw weights $w_p = \Lambda_p p^\alpha$. It also specifies admissible decay profiles in Λ_p when $\alpha \ge -1$ so that (3) still holds.

ACE (safety). ACE enforces the weighted-ℓ_1 budget (3) via a 1-Lipschitz projection (soft-thresholding by a common multiplier), preserving contraction margins. Online metrics: the contraction upper bound $\widehat{\|K\|} = \sum_p b_p|w_p|$, the gap gapLB $= 1 - \widehat{\|K\|}$, and a rate upper bound SlopeUB $= \widehat{\|K\|}$.

7 Algorithms and Reference Implementations

7.1 Exact weighted-ℓ_1 projection (ACE)

Algorithm 1 Weighted-ℓ_1 projection by bisection

Require: raw weights w, PETC bounds b, budget $\tau \in (0,1)$

```
if ∑_i b_i|w_i| ≤ τ then
    return w
end if
Set λ ∈ [0, max_i |w_i|/ max(b_i, ε)]
while budget error > tol and iters < max do
    w_i^proj ← sign(w_i) max(0, |w_i| − λb_i)
    budget ← ∑_i b_i|w_i^proj|
    Adjust λ by bisection to meet budget = τ
end while
return w^proj
```

Listing 1: Exact weighted-ℓ_1 projection (ACE budget)

Python.

```python
from typing import List
import numpy as np

def project_weighted_l1(w: List[float], b: List[float], tau: float,
                        tol: float = 1e-9, max_iters: int = 80) -> List[float]:
    """
    Projects weights to exactly meet ACE budget via bisection.
    Returns projected w satisfying sum(b_i * |w_i|)  tau.
    """
    current_budget = sum(b_i * abs(w_i) for w_i, b_i in zip(w, b))
    if current_budget <= tau:
        return w.copy()

    lo, hi = 0.0, max(abs(w_i) / max(b_i, 1e-15) for w_i, b_i in zip(w, b))
    w_proj = w.copy()
    for _ in range(max_iters):
        lam = 0.5 * (lo + hi)
        w_proj = [
            np.sign(w_i) * max(0.0, abs(w_i) - lam * b_i)
            for w_i, b_i in zip(w, b)
        ]
        budget = sum(b_i * abs(wi) for wi, b_i in zip(w_proj, b))
        if abs(budget - tau) < tol:
            break
        if budget > tau:
            lo = lam
        else:
            hi = lam
    return w_proj
```

7.2 Unified ACE–PETC update (scalar and operator cases)

Listing 2: One PIRTM step with ACE×PETC certification

```python
from typing import List, Callable, Dict, Tuple
import numpy as np

Tensor = np.ndarray
LinearOp = Callable[[Tensor], Tensor]


def ace_petc_step(T: Tensor, F: Tensor, primes: List[int], alpha: float,
                  Lambda: List[float], B_ops: List[LinearOp] = None,
                  b_p: List[float] = None, tau: float = 0.9
                  ) -> Tuple[Tensor, Dict, Dict]:
    """
    One step of PIRTM recurrence with ACEPETC certification.
    Returns: T_next, margins, debug
    """
    w_raw = [Lam_i * (p ** alpha) for p, Lam_i in zip(primes, Lambda)]

    if B_ops is None:
        # Scalar specialization
        b_scalar = b_p if b_p is not None else [1.0] * len(primes)
        w_proj = project_weighted_l1(w_raw, b_scalar, tau)
        k_eff = sum(w_proj)
```

```
        T_next = k_eff * T + F
        margins = {
            "||K||": abs(k_eff),
            "gapLB": 1.0 - abs(k_eff),
            "contraction_rate": abs(k_eff),
            "budget_used": sum(b_i * abs(w_i) for w_i, b_i in zip(w_proj, b_scalar)),
        }
        debug = {"w": w_proj, "k_eff": k_eff, "w_raw": w_raw}
        return T_next, margins, debug

    # Operator case
    assert b_p is not None and len(b_p) == len(primes)
    w_proj = project_weighted_l1(w_raw, b_p, tau)

    def K_operator(X: Tensor) -> Tensor:
        Y = np.zeros_like(X)
        for w_i, B_i in zip(w_proj, B_ops):
            Y = Y + w_i * B_i(X)
        return Y

    T_next = K_operator(T) + F
    K_norm_upper = sum(abs(w_i) * bp for w_i, bp in zip(w_proj, b_p))
    margins = {
        "||K||_ub": K_norm_upper,
        "gapLB": 1.0 - K_norm_upper,
        "contraction_rate": K_norm_upper,
        "budget_used": K_norm_upper,
    }
    debug = {"w": w_proj, "K_norm_ub": K_norm_upper, "K_operator": K_operator}
    return T_next, margins, debug
```

7.3 Monitoring and safety status

Listing 3: Real-time ACE margins monitor

```
class PIRTMMonitor:
    """Real-time monitoring of ACEPETC safety margins"""
    def __init__(self, tau: float = 0.9, window: int = 100, norm_fn=None):
        self.tau = tau
        self.window = window
        self.norm_fn = norm_fn if norm_fn is not None else \
            (lambda x: float(np.linalg.norm(np.asarray(x))))
        self.history = {"K_norm": [], "gapLB": [], "T_diff": [], "sensitivity": []}

    def update(self, margins: Dict, T_prev: Tensor, T_current: Tensor):
        K_norm = margins.get("||K||", margins.get("||K||_ub", 0.0))
        gap = margins.get("gapLB", 0.0)
        self.history["K_norm"].append(K_norm)
        self.history["gapLB"].append(gap)
        self.history["T_diff"].append(self.norm_fn(np.asarray(T_current) - np.asarray(
    T_prev)))
        self.history["sensitivity"].append(1.0 / max(gap, 1e-9))
        for key in self.history:
            if len(self.history[key]) > self.window:
                self.history[key].pop(0)

    def safety_status(self) -> Dict:
```

```
        if not self.history["K_norm"]:
            return {"status": "NO_DATA"}
        recent_K = float(np.mean(self.history["K_norm"][ -5: ]))
        recent_gap = float(np.mean(self.history["gapLB"][ -5: ]))
        if recent_K > self.tau:
            status = "VIOLATION"
        elif recent_gap < 0.05:
            status = "CRITICAL"
        elif recent_gap < 0.10:
            status = "WARNING"
        else:
            status = "SAFE"
        return {
            "status": status,
            "avg_contraction": recent_K,
            "avg_gap": recent_gap,
            "sensitivity": 1.0 / max(recent_gap, 1e-9),
        }
```

7.4 Fixed-point estimator with certified tail

Listing 4: (I - K)$^{-1}FviaNeumannserieswithtailcertificate$

```
def fixed_point_estimate(F: Tensor,
                         K_apply: LinearOp,
                         K_norm_ub: float,
                         tol: float = 1e-8,
                         max_terms: int = 10_000) -> Tensor:
    """Computes (I - K)^{-1} F via Neumann series; requires K_norm_ub < 1."""
    assert K_norm_ub < 1.0, "Neumann series requires ||K|| < 1."
    S = np.zeros_like(F)
    term = F.copy()
    tau = K_norm_ub
    for j in range(max_terms):
        S = S + term
        tail_bound = (tau ** (j + 1)) * float(np.linalg.norm(np.asarray(F))) / (1 -
    tau)
        if tail_bound <= tol:
            break
        term = K_apply(term)
    return S
```

7.5 Adaptive budget to maintain a target gap

Listing 5: Simple adaptive policy for to keep a safety gap

```
def adapt_tau(tau: float, gapLB: float, gap_target: float = 0.1, eta: float = 0.5) ->
    float:
    """If gapLB < target, move tau toward 1 - gap_target by factor eta."""
    if gapLB < gap_target:
        return (1 - eta) * tau + eta * (1.0 - gap_target)
    return tau
```

7.6 Infinite-prime convergence check (corrected)

Listing 6: Absolute convergence test for $\mathrm{b}_p|_p|p$

```python
def check_infinite_prime_convergence(alpha: float, Lambda_bound: float, b_bound: float
    ) -> bool:
    """Returns True iff  < -1 and bounds are finite and nonnegative."""
    ok_bounds = np.isfinite(Lambda_bound) and np.isfinite(b_bound) and (Lambda_bound
    >= 0) and (b_bound >= 0)
    return (alpha < -1) and ok_bounds
```

7.7 Usage example (scalar case)

Listing 7: Scalar PIRTM loop with monitoring and fixed-point estimate

```python
primes = [2, 3, 5, 7, 11]
alpha = -1.5
Lambda = [0.1] * len(primes)
F = 1.0
T = 0.0

monitor = PIRTMMonitor(tau=0.85)
for step in range(100):
    T_next, margins, debug = ace_petc_step(T, F, primes, alpha, Lambda, tau=0.85)
    monitor.update(margins, T, T_next)
    T = T_next
    if step % 10 == 0:
        status = monitor.safety_status()
        print(f"Step {step}: {status['status']} (K={margins.get('||K||', 0):.3f}, gap
    ={margins['gapLB']:.3f})")

# Fixed point
k_eff = debug['k_eff']
T_inf = F / (1 - k_eff)
print(f"Fixed point: {T_inf:.6f}")
```

7.8 Design Guidelines and Checklists

- **Choose** α: start with $\alpha \in (-2, -1)$; adjust Λ_p to keep $\sum b_p|w_p| \leq \tau$.
- **Projection**: always use the exact weighted-ℓ_1 projection (not uniform scaling).
- **Monitoring**: log $\widehat{\|K\|}$, gap $1 - \widehat{\|K\|}$, and $\|T_{t+1} - T_t\|$.
- **Safety margins**: begin with $\tau \leq 0.9$; adapt using `adapt_tau` only when margins erode.
- **Infinite primes**: ensure $\sum_p b_p|\Lambda_p|p^\alpha < 1$ (e.g., $\alpha < -1$ with bounded Λ_p, b_p), noting this is sufficient but not necessary.

7.9 Conclusions

The PIRTM dynamics combined with ACE and PETC yield a contractive, well-conditioned, and monitorable system. The theory (spectral radius criterion, budget-based sufficiency) aligns exactly with the provided implementations (projection, monitors, fixed-point estimation), enabling reliable deployment.

8 Lightweight Scalar Friendly

We study a prime-indexed linear iteration for tensors,

$$T_{t+1} = F + k\,T_t, \quad k \in \mathbb{R},$$

and show that convergence is guaranteed *iff* $|k| < 1$. "Prime structure" is treated as a *dictionary* (representation/regularization) and as *typing metadata* via the **Prime-Encoded Tensor Calculus (PETC; Ryan Van Gelder)** [**?**], rather than as a physical law. Stability and safety are certified with a 1-Lipschitz projection from the **Arithmetic Control Engine (ACE; Tyler Van Osdol)** [**?**]. We replace universal constants with explicit budgets, move speculative physics to future work with a parameter-identification plan, and add a preregistered experimental protocol.

8.1 Notation and Setup

Let $(, \|\cdot\|)$ be a real Banach space (e.g., ℓ_2 coefficients under a fixed dictionary). Fix a finite prime set $P_N = \{p_1, \ldots, p_N\}$. A *prime dictionary* is a collection $= \{\phi_p\}_{p \in P_N} \subset$ used for expansions and penalties; we make no basis/completeness claim. For external drive $F \in$, define the scalar gain

$$k \;=\; \sum_{p \in P_N} \Lambda_p\, p^{\alpha}, \qquad \Lambda_p \in \mathbb{R},\ \alpha \in \mathbb{R}. \tag{4}$$

Older drafts used a single Λ; here we allow per-prime Λ_p.

8.2 Convergence and Safety

Theorem 5 (Convergence $\Leftrightarrow$ contraction)**.** *Let $\mathcal{T}(X) = F + kX$ on . If $|k| < 1$, then for any T_0,*

$$T_t \to T_\infty = \frac{1}{1-k}\,F,$$

with linear rate $|k|$.

Proof. We have $\|\mathcal{T}(X) - \mathcal{T}(Y)\| = |k|\,\|X - Y\|$. If $|k| < 1$, $\mathcal{T}$ is a contraction; apply Banach's fixed-point theorem [**?**]. □

Lemma 6 (One sufficient route to $|k| < 1$)**.** *If $\Lambda_p \geq 0$ and $\alpha < -1$, then*

$$|k| = \sum_{p \in P_N} \Lambda_p p^{\alpha} \;\leq\; \Big(\max_p \Lambda_p\Big) \sum_{i=1}^{N} p_i^{\alpha}.$$

Choosing $\max_p \Lambda_p$ *small enough makes* $|k| < 1$. (Sufficient, not necessary.)

ACE safety wrapper (spectral certification). Let $S \subset$ be a safety/regularization set (e.g., a weighted-ℓ_1 ball on dictionary coefficients). Let Π_S be the Euclidean projection onto S (1-Lipschitz). The ACE-wrapped update

$$T_{t+1} \;=\; \Pi_S\big(F + k\,T_t\big) \tag{5}$$

has global Lipschitz constant $\leq |k|$; thus $|k| < 1$ gives a certified contraction and feasibility $T_t \in S$ for all t.

8.3 PETC Typing: Prime Structure as Metadata

Expand $T = \sum_{p \in P_N} w_p \, \phi_p$. Each coefficient w_p carries a PETC "prime type" tag [**?**]. We impose a *prime-mix budget*

$$\sum_{p \in P_N} b_p \, |w_p(T)| \;\leq\; \tau, \qquad b_p > 0, \tag{6}$$

which is enforced via the projection Π_S (weighted soft-thresholding on w_p). Optional *prime gating* $g_p(\alpha, \Lambda_p) = \Lambda_p p^\alpha$ modulates proposals but is bounded by the same budget; no "universal constant" is assumed.

8.4 Algorithm (ACE×PETC-compliant)

Inputs. F, dictionary $\{\phi_p\}$, weights $\{\Lambda_p\}$, exponent α, budget (b_p, τ).

1. **Gain synthesis.** Compute k from eq. (4). If $|k| \geq 1$, shrink Λ_p and/or P_N until $|k| < 1$.
2. **Projection set.** Define $S = \{T : \sum_p b_p |w_p(T)| \leq \tau\}$. Use Π_S (1-Lipschitz).
3. **Iteration.** $T_{t+1} = \Pi_S(F + kT_t)$ as in eq. (5).
4. **Stopping.** Stop when $\|T_{t+1} - T_t\| \leq \varepsilon$; output T_∞.

Properties. Certified convergence (theorem 5), feasibility (ACE), and auditable prime composition (PETC).

8.5 Practical Guidance

Choosing α, Λ_p, P_N. Pick P_N by model/compute budget; start with $\alpha \in (-2, -1)$ and uniform $\Lambda_p = \Lambda$; use theorem 6 to get a safe Λ with margin $\varepsilon \in [0.1, 0.3]$; tune (b_p, τ) and re-check $|k| < 1$.

Diagnostics. Track $|k|$, budget usage $\sum b_p |w_p|$, and prime-mix histograms per iteration. If convergence slows, reduce $|k|$, tighten τ, or shrink P_N.

8.6 Experimental Protocol (Preregistered)

Benchmarks. MSD-PrimeMix (multi-scale disturbances with controlled prime-spread).
Baselines. LQR (if model known), projected gradient with fixed step, Adam; RL baseline (e.g., TD3/DQN as applicable).
Metrics. Task cost, settling time, safety-violation count, robustness vs. disturbance amplitude/sparsity; report mean±95% CI over 10 seeds.
Ablations. With/without ACE; with/without PETC budget; vary α at fixed $|k|$; expand/shrink P_N.
Reporting. Publish code/configs; include certificate traces (GapLB, SlopeUB).

8.7 Related Work

Contractions and projection operators are classical [**?**]; proximal methods and Lipschitz projections connect to the modern optimization toolkit [**?**]. Weighted ℓ_1 projections and soft-thresholding date to wavelet shrinkage [**?**] and LASSO [**?**]. Prime-indexed dictionaries fit within sparse coding and dictionary learning [**?**, **?**, **?**]. Lipschitz-controlled neural networks relate via Parseval regularization [**?**] and spectral normalization [**?**]. The ACE and PETC frameworks formalize safety via contraction certificates and structural typing, respectively [**?**, **?**].

8.8 Limitations

The linear core is intentional; nonlinear variants require fresh contraction proofs (local or global). Dictionary choice is application-dependent; performance hinges on whether $\{\phi_p\}$ captures salient structure.

8.9 Future Work

Operator learning. Learn diagonal-plus-low-rank operators in the prime dictionary with PETC regularizers; verify a contraction region.
Physical identification. Units-consistent maps from data to (Λ_p, α) with uncertainty quantification.
Nonlinear plants. ACE-style sector-bounded contractions and piecewise certificates.

9 The Prime-Recursive Foundations of Mathematical Existence

A central question is whether diverse structures—algebraic, topological, or computational—can be generated by a single, prime-based, recursive mechanism. We formalize this as a meta-recursive *prime-indexed* constructor that is both *auditable* (PETC: unambiguous retrospection) and *stable* (ACE: certified contraction).

9.1 The Meta-Recursive Function (ACE×PETC Form)

Let $P_N = \{p_1, \ldots, p_N\}$ be the first N primes. Let H be a Banach space of (typed) tensors. For each prime $p \in P_N$ choose a bounded linear channel $B_p : H \to H$ with $\|B_p\| \le b_p$. Define prime weights

$$w_p = \Lambda_p\, p^\alpha, \quad \Lambda_p \in \mathbb{R} \text{ (typically } \Lambda_p \ge 0), \quad \alpha < -1 \text{ (sufficient decay)}.$$

The *gain operator* and the *meta-recursion* are

$$\mathcal{K} = \sum_{p \in P_N} w_p B_p, \qquad T_{t+1} = \mathcal{M}(P_N; T_t) := F + \mathcal{K}\, T_t. \tag{7}$$

This refines the informal sum $\sum_p \Lambda\, p^\alpha T_p^{(m,n)}$ into an operator form while keeping primes explicit as independent channels.

[ACE Contraction Budget] $\sum_{p \in P_N} |w_p|\, b_p \;<\; 1 \quad \Longrightarrow \quad \|\mathcal{K}\| \le \sum_p |w_p| b_p < 1.$

Theorem 7 (Convergence & Fixed Point)**.** *Under Assumption B.2, the affine map $T \mapsto F + \mathcal{K}T$ is a strict contraction on $(H, \|\cdot\|)$. Hence there exists a unique fixed point*

$$T_\infty = (I - \mathcal{K})^{-1} F,$$

and for any T_0 one has linear convergence $\|T_t - T_\infty\| \le \|\mathcal{K}\|^t\, \|T_0 - T_\infty\|$. In the scalar special case ($T \in \mathbb{R}$, $\mathcal{K} = k \in (-1, 1)$) this reduces to $T_\infty = \frac{F}{1-k}$.

Theorem 8 (Uniqueness & Stability under Projection)**.** *Let Π_S be a nonexpansive projection (e.g., a convex projection) and consider the projected recursion*

$$T_{t+1} = \Pi_S\big(F + \mathcal{K}T_t\big).$$

If Assumption B.2 holds, then $T \mapsto \Pi_S(F + \mathcal{K}T)$ is also a contraction with constant $\le \|\mathcal{K}\| < 1$; therefore it admits a unique fixed point in S and the iteration converges linearly to it.

Remark 9 (Safe Design Regime for Prime Weights). A sufficient (not necessary) recipe: choose $\alpha < -1$, $\Lambda_p \ge 0$, and cap $\Lambda_p \le \bar{\Lambda}$ so that $\sum_{p \in P_N} \Lambda_p p^\alpha b_p < 1$. Increasing N or tightening $\bar{\Lambda}$ preserves the margin.

9.2 PETC: Prime-Signature Ledger for Unambiguous Retrospection

Let $\mathbb{P}$ be the set of primes. A *prime signature* is a finitely supported vector $e = (e_p)_{p\in\mathbb{P}} \in \mathbb{Z}^{(\mathbb{P})}$. Define the multiplicity map

$$M(e) = \prod_p p^{e_p} \in \mathbb{Q}_{>0}.$$

Composition of interactions is $e + e'$ (multiset union); undo/consumption is $e - e'$; dualization is $-e$. Unique factorization makes the history canonical: $e_p = v_p(M(e))$ records how many times the p-tagged event occurred.

Definition 10 (Conservation Certificate)**.** For any typed operation with inputs $X_1, \ldots, X_r$ and output Y, PETC conservation requires $e(Y) = \sum_{i=1}^{r} e(X_i)$. A system is *ledger-consistent* if the equality holds for all executed operations.

Proposition 11 (Lossless Retrospection)**.** *If a system is ledger-consistent, then for every state the accumulated signature e determines the exact multiset of prime-tagged interactions that occurred, via $e_p = v_p(M(e))$.*

Complexity. Maintain e as a sparse map (prime $\mapsto$ exponent). Updates/audits run in $O(|\mathrm{supp}(e)|)$—no integer factorization is needed.

9.3 Interpretation as a Universal Constructor

Recursive self-modification. The assignment $T_{t+1} = \mathcal{M}(P_N; T_t)$ mirrors prior states via $\mathcal{K}$, with uniqueness/convergence guaranteed by Theorems 7–8.

Structure emergence. Choosing F (drive) and $\{B_p\}$ (channels) yields a wide class of constructs: neural weights, quantum states, algebraic operators, or kernel process states.

Universality (within H). Iteration converges to $T_\infty = (I - \mathcal{K})^{-1}F$; by selecting F and $\{B_p\}$ appropriately, one can encode a broad family of objects in tensor space.

9.4 Examples of Emergent Structures

1. **Algebraic Systems.** Let B_p be structured projectors/sparsity masks; F encodes a group/semiring operation tensor. Contraction yields stable algebraic updates.

2. **Topological/Geometric Bases.** Choose B_p as basis selectors; prime indices organize connectivity. Stable decompositions follow from Theorem 7.

3. **Tensor Networks.** Let B_p act on tensor cores/edges; the fixed point realizes a high-dimensional network with controlled growth.

4. **Reinforcement Learning.** Take T as weights, F as TD/gradient drive, and B_p as feature filters. Stability ensures convergent Q/policy updates.

5. **Kernel-Level Scheduling.** Model dispatch channels as B_p and resource demand as F for a multiplicity-driven scheduler (Section 9.7).

Theorem 12 (Fractal Stability via Prime-Indexed IFS)**.** *If $H = \prod_j H_j$ with componentwise contractions $\{T \mapsto F + \mathcal{K}T\}$ whose constants are < 1, then the induced self-map on H is a contraction; the unique attractor T_∞ is the limit of a prime-indexed iterated function system (IFS). Hierarchical prime partitioning induces self-similar structure in the limit.*

9.5 Refinements: Compression, Hierarchies, Causality, and Performance

Signature compression (checkpointing). At epoch k, store $c_k \leftarrow e$ and reset $e \leftarrow 0$. The rolled signature $\overline{e} = \sum_k c_k + e$ equals the uncheckpointed signature, preserving audits exactly.

Exact/approximate/log-space conservation. Prefer exact exponent equality $\sum_i e(X_i) = e(Y)$. For noise, allow $\| \sum_i e(X_i) - e(Y) \|_\infty \le \varepsilon$. If only $M(e)$ is accessible, use $\sum_i \sum_p e_p(X_i) \log p \approx \sum_p e_p(Y) \log p$ with tolerance η.

Hierarchical rollups. For a module tree $\mathcal{T}$ with local signatures $e[u]$, define $E[u] = e[u] + \sum_{v \in \mathrm{Children}(u)} E[v]$. If primitives are conservative, then $E[u]$ matches u's external interface; conservation at the root implies global conservation.

Causality (optional). Augment PETC with a DAG $G = (V, E)$; label each edge with its prime increment. The justification set for a node (state) is the multiset union along any source–node path; its signature equals the state's PETC signature.

9.6 ACE: Exact Weighted-ℓ_1 Projection

We enforce $\sum_p b_p |w_p| \le \tau < 1$ via the exact projection onto the weighted ball. Let $z_p = b_p |w_p|$. If $\sum_p z_p \le \tau$ return w; else find $\theta \ge 0$ with $\sum_p \max(z_p - \theta, 0) = \tau$ and set

$$w_p^\star = \mathrm{sign}(w_p) \, \frac{\max(z_p - \theta, 0)}{b_p}.$$

This is the weighted analogue of soft-thresholding; θ can be found by sorting $\{z_p\}$ and a cumulative scan.

9.7 Kernel-Level Multiplicity Scheduler

Associate each queue/class with a prime p; maintain PETC signatures per process and per core. Use ACE to allocate per-prime weights w_p under $\sum b_p |w_p| \le \tau < 1$; update dispatch decisions via $T_{t+1} = F + \mathcal{K} T_t$ where T aggregates runnable-state indicators and F encodes exogenous demand. PETC provides exact provenance for merges/splits; ACE maintains stability and responsiveness through budgeted adjustments.

9.8 Algorithms and Code (Runnable Sketches)

PETC prime ledger with checkpointing and exact conservation.

```
class PrimeLedger:
    def __init__(self):
        self.sig = {}          # dict[int -> int], sparse exponents e_p
        self.checkpoints = []  # list[dict[int -> int]]

    def add_event(self, p: int, mult: int = 1, sign: int = +1):
        self.sig[p] = self.sig.get(p, 0) + sign * mult
        if self.sig[p] == 0: self.sig.pop(p)

    def checkpoint(self):
        self.checkpoints.append(self.sig.copy())
        self.sig.clear()

    def rolled_signature(self):
        roll = {}
        def add_map(dst, src, sgn=+1):
            for p, e in src.items():
                dst[p] = dst.get(p, 0) + sgn * e
```

```
                if dst[p] == 0: dst.pop(p)
        for ck in self.checkpoints: add_map(roll, ck, +1)
        add_map(roll, self.sig, +1)
        return roll

def conservation_ok_exact(inputs, out) -> bool:
    acc = {}
    def add(dst, src, sgn=+1):
        for p, e in src.items():
            dst[p] = dst.get(p, 0) + sgn * e
            if dst[p] == 0: dst.pop(p)
    for L in inputs: add(acc, L.rolled_signature(), +1)
    add(acc, out.rolled_signature(), -1)
    return len(acc) == 0
```

Optional log-space surrogate (tolerant).

```
import math

def log_signature(sig: dict[int,int]) -> float:
    return sum(e * math.log(p) for p, e in sig.items())

def conservation_ok_log(inputs, out, eta: float = 1e-9):
    lhs = sum(log_signature(L.rolled_signature()) for L in inputs)
    rhs = log_signature(out.rolled_signature())
    return abs(lhs - rhs) <= eta
```

ACE exact weighted-ℓ_1 projection and prime-indexed operator.

```
import math
import numpy as np

def project_weighted_l1(w: dict[int,float], b: dict[int,float], tau: float):
    z = {p: b[p] * abs(w[p]) for p in w}
    S = sum(z.values())
    if S <= tau: return w
    items = sorted(z.items(), key=lambda kv: kv[1], reverse=True)
    cumsum, theta = 0.0, 0.0
    for k, (_, zk) in enumerate(items, start=1):
        cumsum += zk
        theta = (cumsum - tau) / k
        if k == len(items) or items[k][1] <= theta < items[k-1][1]:
            break
    w_star = {}
    for p in w:
        val = max(z[p] - theta, 0.0) / max(b[p], 1e-12)
        w_star[p] = math.copysign(val, w[p])
    return w_star

class PrimeAffine:
    def __init__(self, B_ops: dict[int, callable], b_norms: dict[int,float], tau: float = 0
        self.B = B_ops     # p -> H->H
```

```
        self.b = b_norms   # p -> ||B_p||
        self.tau = tau
        self.w  = {p: 0.0 for p in B_ops}

    def set_weights(self, Lambda: dict[int,float], alpha: float):
        w = {p: Lambda.get(p, 0.0) * (p ** alpha) for p in self.B}
        self.w = project_weighted_l1(w, self.b, self.tau)

    def K_apply(self, T):
        acc = 0 * T
        for p, Bp in self.B.items():
            acc = acc + self.w[p] * Bp(T)
        return acc

    def step(self, T, F):
        return F + self.K_apply(T)
```

End-to-end loop (with PETC logging and ACE budget check).

```
def run_loop(T0, F, prime_affine: PrimeAffine, ledger: PrimeLedger,
             prime_of_update: int, steps: int = 100):
    T = T0
    for t in range(steps):
        ledger.add_event(prime_of_update, mult=1, sign=+1)   # PETC log
        T_next = prime_affine.step(T, F)                     # ACE step
        budget = sum(abs(prime_affine.w[p]) * prime_affine.b[p] for p in prime_affine.w)
        assert budget < 1.0, "ACE budget violated: not a contraction"
        T = T_next
    return T
```

9.9 Operational Checklist

1. **Ledger (PETC).** Assign primes to components/events; maintain signatures additively; enforce conservation. Use checkpointing and hierarchical rollups for scale; optionally attach a causality DAG.

2. **Weights (ACE).** Set $w_p = \Lambda_p p^\alpha$ with $\alpha < -1$ (sufficient) and project onto $\sum b_p |w_p| \leq \tau < 1$ via weighted-ℓ_1 projection.

3. **Certificates.** Monitor budget $\sum b_p |w_p|$, and (optionally) gap/slope margins during learning.

4. **Convergence.** By Theorems 7–8, $T_{t+1} = F + \mathcal{K}T_t$ (or its projected form) converges uniquely to a fixed point at rate $\|\mathcal{K}\|$.

A Handy Design Bound for $|k|$

For uniform $\Lambda_p = \Lambda$ and $\alpha < -1$,

$$|k| = \Lambda \sum_{i=1}^{N} p_i^\alpha.$$

Solve $|k| = 1 - \varepsilon$ for Λ to initialize with margin ε.

Reproducibility Checklist (suggested). Provide code/configs; fix seeds; publish CSV logs for $|k|$, budget usage, and certificate traces; include README with environment and commit hash.

B Prime-Recursive Mechanical Stabilization: PETC × ACE Formalization

Attribution. ACE (Arithmetic Control Engine): Tyler Van Osdol. PETC (Prime-Encoded Tensor Calculus): Ryan Van Gelder.

B.1 Prime-Signature Ledger (PETC) for Unambiguous Retrospection

Let $\mathbb{P}$ denote the set of primes. A *prime signature* is a finitely supported vector $e \in \mathbb{Z}^{(\mathbb{P})}$; we write $e = (e_p)_{p\in\mathbb{P}}$ with $e_p = 0$ for all but finitely many p. Define the *multiplicity map*

$$M(e) \;=\; \prod_{p\in\mathbb{P}} p^{e_p} \;\in\; \mathbb{Q}_{>0}.$$

Operations are componentwise: composition of events is $e + e'$, undo/consumption is $e - e'$, dualization is $-e$. This realizes the slogan "*multiply primes now, factor later*" as *add exponents now, read them later*: unique factorization ensures that e (hence the full interaction multiset) is recoverable without ambiguity.

Definition 13 (Conservation Certificate)**.** For any typed operation Φ with inputs $X_1, \ldots, X_r$ and output Y, the PETC *conservation check* requires $e(Y) \;=\; \sum_{i=1}^{r} e(X_i)$. A system is *ledger-consistent* if the equality holds for all executed Φ.

Proposition 14 (Lossless Retrospection)**.** *If a system is ledger-consistent, then for every state the accumulated signature e determines the exact multiset of prime-tagged interactions that occurred (with multiplicities), via the valuation $e_p = v_p(M(e))$.*

Complexity. Maintain e as a sparse map (prime $\mapsto$ exponent). Updates and audits are $O(|\mathrm{supp}(e)|)$; no integer factorization is required.

B.2 Prime-Indexed Affine Recursion (ACE-Certified)

Let H be a Banach space of tensors (axes carry PETC types). For each prime p choose a bounded linear "channel" $B_p : H \to H$ with $\|B_p\| \le b_p$. Choose per-prime weights $w_p = \Lambda_p\, p^{\alpha}$ (design knob: $\Lambda_p \in \mathbb{R}$, typically $\Lambda_p \ge 0$, and $\alpha < -1$ for decay). Define the gain operator

$$\mathcal{K} \;=\; \sum_{p\in P_N} w_p\, B_p, \qquad \|\mathcal{K}\| \;\le\; \sum_{p\in P_N} |w_p|\, b_p.$$

The *meta-recursion* is the affine map

$$T_{t+1} \;=\; F \;+\; \mathcal{K}\, T_t, \tag{8}$$

which collapses your original sum into an operator form while keeping primes as explicit channels.

[ACE Contraction Budget] $\displaystyle\sum_{p\in P_N} |w_p|\, b_p \;<\; 1.$

Theorem 15 (Fixed Point, Uniqueness, and Linear Rate)**.** *Under Assumption B.2, the recursion (8) is a strict contraction on $(H, \|\cdot\|)$. Hence there exists a unique fixed point $T_\infty = (I - \mathcal{K})^{-1} F$ and for any T_0*

$$\|T_t - T_\infty\| \;\le\; \|\mathcal{K}\|^{\,t}\, \|T_0 - T_\infty\|.$$

In the scalar special case $T \in \mathbb{R}$ and $\mathcal{K} = k \in (-1, 1)$, $T_\infty = \frac{F}{1-k}$.

Remark 16 (Safe Design Regime). A sufficient (not necessary) recipe is $\alpha < -1$, $\Lambda_p \geq 0$, and a global cap $\Lambda_p \leq \bar{\Lambda}$ chosen so that $\sum_{p \in P_N} \Lambda_p p^\alpha b_p < 1$. Increase N or tighten $\bar{\Lambda}$ to keep the margin.

ACE Safety Certificates. Let $\delta_S > 0$ be a certified lower bound on the system's stability gap in the absence of $\mathcal{K}$, and let L_p bound the local schedule slope contributed by channel B_p. Define

$$\mathrm{GapLB}(w) := \delta_S - 2 \sum_{p \in P_N} b_p |w_p|, \qquad \mathrm{SlopeUB}(w) := \sum_{p \in P_N} L_p |w_p|.$$

Operate under the constraints $\mathrm{GapLB}(w) > 0$ and $\mathrm{SlopeUB}(w) < 1$. These provide online, quantitative margins for stability and non-expansiveness during learning.

B.3 Algorithms and Code Snippets

PETC Prime Ledger (lossless history).

```
class PrimeLedger:
    def __init__(self):
        self.sig = {}  # dict[int -> int], sparse exponents e_p

    def add_event(self, p: int, mult: int = 1, sign: int = +1):
        # sign=+1 for occurrence, -1 for consumption/undo
        self.sig[p] = self.sig.get(p, 0) + sign * mult
        if self.sig[p] == 0:
            self.sig.pop(p)

    def valuation(self, p: int) -> int:
        return self.sig.get(p, 0)

    def multiplicity_Q(self):
        # returns numerator, denominator of product p^{e_p}
        num, den = 1, 1
        for p, e in self.sig.items():
            if e >= 0: num *= p ** e
            else:      den *= p ** (-e)
        return num, den

def conservation_ok(inputs, output) -> bool:
    # Check PETC conservation: sum(e_in) == e_out
    acc = {}
    def add_into(dst, src, sgn=+1):
        for p, e in src.sig.items():
            dst[p] = dst.get(p, 0) + sgn * e
            if dst[p] == 0: dst.pop(p)
    for L in inputs: add_into(acc, L, +1)
    add_into(acc, output, -1)
    return len(acc) == 0
```

ACE-Certified Affine Update with Budget Projection. We implement a simple (conservative) weighted-ℓ_1 shrinkage to enforce the budget $\sum_p b_p |w_p| \leq \tau < 1$. Replace `shrink_wl1` with an exact weighted-ℓ_1 projection if needed.

```
import numpy as np

def shrink_wl1(w, b, tau):
    # Conservative shrink: scale if the budget is exceeded
    # w, b are dict[int->float]. Returns dict[int->float].
    budget = sum(abs(w[p]) * b[p] for p in w)
    if budget <= tau: return w
    scale = tau / budget
    return {p: scale * w[p] for p in w}

class PrimeAffine:
    def __init__(self, B_ops, b_norms, tau=0.9):
        # B_ops: dict[int -> callable H->H], b_norms: dict[int->float] with ||B_p||<=b_p
        self.B = B_ops
        self.b = b_norms
        self.tau = tau
        self.w  = {p: 0.0 for p in B_ops}  # start neutral

    def set_weights(self, Lambda, alpha):
        # Lambda: dict[int->float], alpha< -1 recommended
        w = {p: Lambda.get(p, 0.0) * (p ** alpha) for p in self.B}
        self.w = shrink_wl1(w, self.b, self.tau)  # enforce ACE budget

    def K_apply(self, T):
        acc = 0 * T
        for p, Bp in self.B.items():
            acc = acc + self.w[p] * Bp(T)
        return acc

    def step(self, T, F):
        # One affine contraction step: T_{t+1} = F + K T_t
        return F + self.K_apply(T)
```

End-to-End Loop with Ledger and Stability Checks.

```
def run_loop(T0, F, prime_affine, ledger, prime_of_update, steps=100):
    T = T0
    for t in range(steps):
        # Log the update event in PETC:
        ledger.add_event(prime_of_update, mult=1, sign=+1)

        # ACE contraction step:
        T_next = prime_affine.step(T, F)

        # Optional: online margin monitoring
        budget = sum(abs(prime_affine.w[p]) * prime_affine.b[p] for p in prime_affine.w)
        assert budget < 1.0, "ACE budget violated: not a contraction"

        T = T_next
    return T
```

B.4 Design Pattern and Examples

- **RL / Control.** Let T be a weight tensor (e.g., linear Q or policy params), F the external drive (TD target / gradient), and B_p prime-typed feature filters. PETC logs every gradient/apply step; ACE ensures $\sum b_p|w_p| < 1$ for stable convergence to T_∞.
- **Tensor Networks / Algebraic Systems.** Choose B_p as structured projectors or sparsity masks whose norms are known. PETC ensures typed contractions respect multiplicity conservation; ACE budgets keep the build-up stable.
- **Auditing & Replay.** Given a terminal signature e, valuations e_p give exact interaction counts (multiset multiplicities). The system can replay or attribute decisions without ambiguity.

B.5 Takeaways (Operational Rules)

1. **Ledger:** assign each component/event a prime; update signatures additively; enforce PETC conservation.
2. **Weights:** set $w_p = \Lambda_p p^\alpha$ with $\alpha < -1$ (sufficient) and cap Λ_p to satisfy $\sum |w_p| b_p < 1$.
3. **Certificates:** monitor $\mathrm{GapLB}(w) > 0$ and $\mathrm{SlopeUB}(w) < 1$ online.
4. **Convergence:** $T_{t+1} = F + \mathcal{K}T_t$ contracts to $T_\infty = (I - \mathcal{K})^{-1}F$ with linear rate $\|\mathcal{K}\|$.